Topic: CNN Architecture — LeNet-5

Introduction:

Before deep architectures like ResNet, VGG, or Inception, there was **LeNet-5**, one of the **earliest successful Convolutional Neural Networks**, developed by **Yann LeCun in 1998**.
It was used to recognize **handwritten digits** (like those in the MNIST dataset — 0–9).
LeNet-5 proved that neural networks could automatically learn image features — without manual feature extraction.
That discovery changed computer vision forever.

CNN Architecture Recap

Let's quickly recall what a general CNN architecture looks like:

1. **Input Layer** — Raw pixel values of the image (e.g., 32×32×1 for grayscale).
2. **Convolutional Layers** — Detect patterns (edges, textures).
3. **Activation Functions (ReLU/Tanh)** — Introduce non-linearity.
4. **Pooling Layers** — Reduce spatial size and retain main features.
5. **Flatten Layer** — Converts 2D feature maps into 1D vector.
6. **Fully Connected Layers** — Combine features and make final decision.
7. **Output Layer** — Gives class probabilities (e.g., which digit it is).

A CNN learns from simple → to complex:

- Early layers detect **edges**
- Middle layers detect **shapes or textures**
- Final layers detect **objects or digits**

LeNet-5 Architecture

Let's dive into the LeNet-5 step by step.

1 Input Layer

- **Input:** 32 × 32 grayscale image (1 channel)
- Example: a handwritten digit

2 C1 — First Convolution Layer

- **Operation:** 6 filters (kernels) of size 5×5
- **Stride:** 1
- **Output:** 28 × 28 × 6 feature maps

How?

→ (32 - 5 + 1) = 28 (for each dimension)

Purpose: Detect basic edges, corners, textures.

---

3 S2 — Subsampling (Pooling) Layer

- **Operation:** Average Pooling with 2×2 window
- **Stride:** 2
- **Output:** 14 × 14 × 6

Purpose: Reduce size & make features position-invariant.

---

4 C3 — Second Convolution Layer

- **Operation:** 16 filters of size 5×5
- **Input:** 14 × 14 × 6
- **Output:** 10 × 10 × 16

Purpose: Learn more complex shapes and patterns.

---

5 S4 — Subsampling (Pooling) Layer

- **Operation:** Average Pooling again (2×2 window, stride 2)
- **Output:** 5 × 5 × 16

Purpose: Compress and keep strong activations.

---

6 C5 — Fully Connected Convolutional Layer

- **Operation:** 120 filters, each of size 5×5
- **Input:** 5 × 5 × 16
- **Output:** 1 × 1 × 120

Note: Since the input is exactly 5×5, each filter covers the entire input — this acts like a fully connected layer.

Purpose: Learn high-level features for classification.

---

7 F6 — Fully Connected Layer

- **Neurons:** 84
- **Activation:** Tanh (in original LeNet)
- Connected to all 120 neurons from the previous layer.

Purpose: Combine all learned features to form decision signals.

---

8 Output Layer

- **Neurons:** 10 (for digits 0–9)
- **Activation:** Softmax (to output probabilities)

Purpose: Final prediction of which digit the image represents.

Summary Table: LeNet-5 Layers

Layer	Type	Input Size	Kernel	Stride	Output Size	Activation
C1	Convolution	32×32×1	5×5	1	28×28×6	tanh
S2	Average Pooling	28×28×6	2×2	2	14×14×6	tanh
C3	Convolution	14×14×6	5×5	1	10×10×16	tanh
S4	Average Pooling	10×10×16	2×2	2	5×5×16	tanh
C5	Convolution	5×5×16	5×5	1	1×1×120	tanh
F6	Fully Connected	120	—	—	84	tanh
Output	Fully Connected	84	—	—	10	softmax

Code Example (Keras Implementation)

Here's how LeNet-5 looks in **Keras**:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, AveragePooling2D, Flatten, Dense

model = Sequential()

# Layer 1: Convolution
model.add(Conv2D(6, kernel_size=(5,5), activation='tanh', input_shape=(32,32,1)))

# Layer 2: Average Pooling
model.add(AveragePooling2D(pool_size=(2,2)))

# Layer 3: Convolution
model.add(Conv2D(16, kernel_size=(5,5), activation='tanh'))

# Layer 4: Average Pooling
model.add(AveragePooling2D(pool_size=(2,2)))

# Layer 5: Convolution (Fully connected equivalent)
model.add(Conv2D(120, kernel_size=(5,5), activation='tanh'))
```

```
# Flatten and Fully Connected Layers
model.add(Flatten())
model.add(Dense(84, activation='tanh'))
model.add(Dense(10, activation='softmax'))

model.summary()
```

This is almost identical to the original LeNet-5 but in modern syntax.

Outro / Key Takeaways

Concept	Explanation
LeNet-5	One of the first CNN architectures for image recognition.
Invented by	Yann LeCun (1998).
Main Use	Handwritten digit recognition (MNIST).
Core Idea	Automatic feature extraction via convolution + pooling.
Activation	Tanh (original), but ReLU used in modern versions.
Pooling	Average pooling used (modern CNNs use Max pooling).
Impact	The foundation for all modern CNNs like AlexNet, VGG, ResNet.

Simple Analogy

Think of LeNet-5 as an "image detective":

- First, it looks for **edges (C1)**
- Then for **shapes (C3)**
- Then it **summarizes (pooling)**
- Finally, it **decides what it sees (F6 & Output)**

Each layer specializes in a level of understanding — just like how your eyes and brain process images step by step.
